Complete Step-by-Step: Zero to Live Site

A literal, in-order procedure. Do each numbered step, confirm it before moving to the next. This assumes zero coding knowledge — every step tells you exactly what to click, sign up for, or say to Claude Code.

STEP 1 — Get Your Tools (one-time, ~30 minutes)

1. **Install Claude Code Desktop.** This is the AI that will write and run all your code.
2. **Create a free GitHub account** at github.com. This is where your project's code is stored and backed up. You'll just need an email and password.
3. **Have your architecture doc ready** — `booking-marketplace-architecture.md`. You'll attach this at the start of your Claude Code project.

Don't do yet: don't buy hosting, a domain, or sign up for Twilio/SendGrid yet. Those come later and cost money to sit idle.

STEP 2 — Start Your Project

1. Open Claude Code, start a new project, and give it a name (e.g. "booking-marketplace").
2. Attach or paste in your `booking-marketplace-architecture.md` file.
3. Tell it: *"This is the architecture for my project. We'll build it in the order listed in Section 8. Let's start with step 1: auth + business onboarding. Set up the project (Next.js frontend, NestJS backend, PostgreSQL database, all runnable locally), then build sign-up/login with roles, and a business onboarding flow for a single location. Tell me exactly how to run it and what to click to test it."*
4. Let it work. It will likely ask you to run a few terminal commands it gives you, or it may run them for you — follow along, and if something errors, just paste the error back and say "this happened, please fix it."

Confirm before moving on: you can open the app in your browser, sign up, log in, and create a business.

STEP 3 — Build Each Remaining Core Feature, One at a Time

Repeat this pattern for each item, in this exact order. Each time:

- Say: *"Make a copy to work on first."* (this is the branching habit)
- Describe the step using the relevant section of the architecture doc.
- After it's built, test it yourself in the browser.
- Say: *"Looks good, commit this and merge it in."* — or — *"This broke X, please fix it before we continue."*

Order	What to ask for	How you personally confirm it works
3a	Service & availability management	Add a service with a price/duration; set weekly hours for a staff member
3b	Public booking flow	Book a real slot as a test consumer; try booking the same slot twice in two browser tabs — it must reject the second one
3c	Notifications/reminders	You actually receive a test confirmation (email/SMS) after booking
3d	Mark-as-completed + Sale recording	Mark a booking "completed & paid," check tax is calculated correctly
3e	Multi-location support	Add a second location, assign a staff member to it, confirm bookings go to the right one
3f	Tax configuration	Change the tax rate, confirm it updates on the next Sale
3g	Analytics dashboard	Check total sales, appointment counts, and per-employee breakdown match what you actually entered
3h	Search & discovery	Search by city/category and find the businesses you created
3i	Reviews	Leave a review on a completed booking, see it show up on the listing
3j	Marketing & CRM	Confirm a customer record was auto-created from a booking; send yourself a real test campaign email/SMS
3k	Admin panel	Approve/reject a test business as the platform admin

Security checkpoint after 3a (do this once, early): tell Claude Code: *"Before we go further — add rate limiting to login, signup, and booking; make sure API keys are never hardcoded, only environment variables; and confirm every endpoint checks that a business owner can only see their own business's data. Test this last part by trying to view a different business's dashboard by editing the URL."*

STEP 4 — Add Realistic Test Data

Ask: *"Add a handful of realistic fake businesses, services, staff, and bookings so I can test search, analytics, and the dashboard with something that isn't empty."* This makes steps 3g–3j actually meaningful to look at.

STEP 5 — Set Up Safety Nets Before Going Live

1. **Automated tests for the critical flows.** Ask: *"Write automated tests confirming: no double-booking is possible, tax is calculated correctly, marketing never sends to opted-out customers, and cancelled bookings never count as completed sales. Run them and confirm they pass."*
2. **Changelog.** Ask: *"Start a CHANGELOG.md and keep it updated automatically with every future change."*
3. **Dependency security audit.** Ask: *"Run a security audit on all dependencies and fix anything critical before we go further."*

STEP 6 — Get a Real Database and Hosting (this is where money starts being spent)

1. **Sign up for a managed Postgres provider** — Supabase or Neon, both have a free tier to start. Just create the account; ask Claude Code to handle the technical connection.
2. Tell Claude Code: *"Migrate our local database to [Supabase/Neon], set up the connection using environment variables, and confirm everything still works pointed at the real database."*
3. **Sign up for a hosting platform** for your frontend and backend — Vercel is the simplest for the Next.js frontend; Render or Railway can host both frontend and backend together. Pick one and tell Claude Code which.
4. Tell it: *"Connect our GitHub repo to [chosen platform] and deploy the app. Set up all required environment variables there — walk me through exactly what to paste in and where."*
5. **Set up a staging deployment too**, before touching production further: *"Set up a second deployment (staging) using a separate test database, so future changes get tested there before going to the live version."*

Confirm before moving on: your app is reachable at a real URL (something like `your-project.vercel.app`), from your phone, not just your laptop.

STEP 7 — Buy a Domain and Connect Security

1. **Buy a domain name** (Namecheap, Squarespace Domains, etc.) — just the purchase, a few dollars/year.
2. Tell Claude Code: *"Help me point this domain at our hosting platform"* — it will give you exact DNS records to enter on your domain registrar's site.
3. **Sign up for Cloudflare (free tier)** and route your domain's DNS through it — this adds DDoS protection and a basic firewall with no code changes. Ask Claude Code to confirm this is set up correctly once you've done it.

4. **Confirm HTTPS is active** — visit your real domain and check for the padlock icon in the browser. This is usually automatic once the domain is connected.

STEP 8 — Set Up Real Communications

1. **Sign up for Twilio** (SMS) — note this can take a few days for ID verification, so do this a week or more before you plan to launch.
2. **Sign up for SendGrid or Postmark** (email).
3. Tell Claude Code: *"Connect our real Twilio and SendGrid accounts using environment variables, and send me a real test booking confirmation and a real test marketing email so I can confirm they actually arrive."*

STEP 9 — The Non-Code Groundwork

1. **Privacy policy & terms of service** — use a reputable free/low-cost generator (Termly or similar); make sure it explicitly covers SMS/email marketing and opt-out, since you're storing customer contact data.
2. **Support contact** — set up a simple support@yourdomain.com email.
3. **Admin account security** — enable 2FA on your own admin account specifically, since it's the highest-value target.
4. **Backups confirmed on** — log into your Supabase/Neon dashboard and confirm daily automatic backups are actually turned on, not just assumed.

STEP 10 — Soft Launch (Real Businesses, Small Scale)

1. Personally onboard 2–5 real local businesses you know or can approach directly. Walk them through it yourself.
2. Let them use it for real appointments for 1–2 weeks.
3. Watch closely for bugs, confusing steps, and feature requests you didn't anticipate.
4. For every fix or change during this period, follow the safe-iteration pattern: work on a copy, test on staging, confirm the core flows (booking, sale/tax, opt-out) still work, then push live.

STEP 11 — Public Launch

1. Once the soft launch has gone a couple of weeks without major issues, expand outreach to more businesses in your target city/category.
2. Confirm error monitoring (Sentry) is actually catching and alerting you to problems — ask Claude Code to set this up if it hasn't already, and trigger a fake error to confirm you get notified.
3. Keep shipping small, tested improvements based on real feedback rather than guessing at big new features.

Quick Reference: The One Sentence to Repeat at Every Step

> *"Make a copy to work on first, only touch what's needed, run the existing tests, tell me what to click through to verify it before we put it live."*

This single sentence, reused at every step from here to launch and beyond, is what keeps the whole project stable as it grows — not because you're checking the code, but because you're consistently asking the AI to check itself before you trust the result.

Ready to start? Say the word and I'll write out the exact opening message for Step 2 (starting your Claude Code project) so you can copy-paste it in right now.